MedClaim AI

AI-Powered Medical Claim Adjudication System

IISc Hackathon 2026 — Problem Statement 1

Team MedClaim

Tejas Bharadhwaj | Samarth Sharma | Jeevith Manjunatha | Yuvan

Packages Solved: MG064A | SG039C | MG006A | SB039A

1. Problem Statement

Health insurance claim adjudication in India is a predominantly manual process prone to delays, inconsistencies, and fraud. Claim officers must review stacks of medical documents — discharge summaries, investigation reports, operative notes, implant invoices — and cross-verify them against Standard Treatment Guidelines (STG) for each procedure package.

The IISc Hackathon Problem Statement 1 requires participants to build an automated system that, for each page of an uploaded claim document, produces a structured JSON output capturing document presence, clinical condition flags, sign classifications, and chronological document ranking — across four procedure packages:

Package Code	Condition	Key Documents Required
MG064A	Severe Anemia	Clinical notes, CBC/Hb report, Indoor case, Treatment details, Post-Hb report, Discharge summary
SG039C	Cholecystectomy	Clinical notes, USG report, LFT report, Operative notes, Pre-anaesthesia, Discharge summary, Photo evidence, Histopathology
MG006A	Enteric Fever	Clinical notes, Pre-investigation + date, Vitals/treatment, Post-investigation + date, Discharge summary
SB039A	Total Knee Replacement	Clinical notes, X-ray/CT knee, Indoor case, Operative notes, Implant invoice, Post-op photo, Post-op X-ray, Discharge summary

2. System Architecture

MedClaim AI is a full-stack web application with a React frontend and a Supabase backend. The pipeline converts uploaded documents into per-page images, sends them to a vision AI model for field extraction, runs a deterministic rules engine against STG criteria, and returns a structured JSON verdict.

Step	Component	Description
1	Document Upload (React + PDF.js)	User uploads PDFs or images. PDF pages are rasterised to JPEG data-URLs client-side using pdfjs-dist at 1.6x scale for readability.
2	Supabase Storage + Database	Files stored in Supabase Storage bucket. A claims record (id, status, file_names, data) is created in the claims table.
3	Vision AI Extraction (Claude Sonnet)	Each page image is sent to the vision model with a structured prompt. The model returns doc_type, confidence, visual_cues (stamp, signature, sticker, QR), and extracted_fields per page.
4	Rules Engine (rules-config.ts)	Deterministic rules run on extracted data: chronology checks (R_CHRONO), document presence (R_DOC), diagnostic requirements (R_DIAG), length-of-stay (R_LOS), authentication (R_AUTH), implant sticker (R_IMPLANT), key-info extraction (R_KIE).
5	JSON Output + Verdict	Per-page JSON is assembled with all required keys per package. A final Pass / Conditional / Fail verdict is computed from weighted rule scores.
6	Chat Interface (AI Chatbot)	An interactive chatbot backed by chat_messages table allows claim officers to query specific findings with source-traced evidence.

2.1 Tech Stack

Layer	Technology
Frontend	React 18, TanStack Router, Tailwind CSS, shadcn/ui
PDF Processing	pdfjs-dist (client-side rasterisation to JPEG)
Backend / DB	Supabase (PostgreSQL, Row-Level Security, Storage)
AI / Vision	Claude Sonnet — vision + structured JSON extraction
Rules Engine	TypeScript (rules-config.ts) — deterministic STG checks
Auth	Supabase Auth — Bearer token middleware (auth-middleware.ts)
Hosting	Lovable Cloud (Vite + TanStack Start SSR)

3. How It Works — Step by Step

Step 1 — Upload & Rasterise

The user visits the MedClaim AI web interface and uploads one or more claim documents (PDF or image). The *filesToPages()* function in **pdf-to-images.ts** rasterises each PDF page to a JPEG data-URL using pdfjs at 1.6x scale. Each page becomes a *ClaimPage* object carrying a unique *doc_id*, *file_name*, *page* number, and *image_url*.

Step 2 — AI Field Extraction

Every page image is sent to the Claude Sonnet vision model. The model is prompted to classify the document type (e.g. *discharge_summary*, *lab_report*, *angioplasty_report*) and extract structured fields: patient name, admission date, procedure date, discharge date, procedure name, and visual cues such as hospital stamp, doctor signature, implant sticker, and QR/barcode presence.

Step 3 — Rules Engine

The **rules-config.ts** module runs eight deterministic checks against the extracted data, each mapped to an STG criterion:

Rule ID	Severity	Check
R_CHRONO_001	High	Admission date ≤ Procedure date
R_CHRONO_002	High	Discharge date ≥ Admission date
R_DOC_001	High	At least one clinical document present
R_DIAG_001	Medium	Diagnostic document present when procedure listed
R_LOS_001/2/3	High/Med	Length of stay ≥ 0; flags zero-day and >30-day stays
R_AUTH_001	Low	Doctor signature on clinical document
R_IMPLANT_001	Medium	Implant sticker present for stent/valve/prosthesis procedures
R_KIE_001	High	Patient name extractable from at least one document

Step 4 — Verdict & JSON Assembly

The *decide()* function computes a weighted score: high-severity fail = 3 pts, medium fail = 2 pts, warn = 1 pt. A high-severity fail always yields **Fail**; score ≥ 2 yields **Conditional**; otherwise **Pass**. The per-page JSON is then assembled with all mandatory keys for the relevant package code.

Step 5 — Timeline & Chatbot

The *buildTimeline()* function sorts all extracted dates into a chronological event sequence (Admission → Investigation → Procedure → Discharge) and flags invalid orderings. The integrated AI chatbot (backed by the *chat_messages* Supabase table) allows adjudicators to ask natural-language questions about any finding with full source tracing back to the originating document and page.

4. Sample JSON Outputs

4.1 Package MG064A — Severe Anemia

Two-page claim: page 1 contains a CBC/Hb report satisfying severe anemia criteria; page 2 is an unrelated consent form (extra document).


```
[
{
"case_id": "C1",
"S3_link": "medclaim-bucket/anemia_claim.pdf",
"procedure_code": "MG064A",
"page_number": 1,
"clinical_notes": 1,
"cbc_hb_report": 1,
"indoor_case": 1,
"treatment_details": 0,
"post_hb_report": 0,
"discharge_summary": 0,
"severe_anemia": 1,
"common_signs": 1,
"significant_signs": 0,
"life_threatening_signs": 0,
"extra_document": 0,
"document_rank": 2
},
{
"case_id": "C2",
"S3_link": "medclaim-bucket/anemia_claim.pdf",
"procedure_code": "MG064A",
"page_number": 2,
... all clinical keys: 0,
"extra_document": 1,
"document_rank": 99
}
]
```


4.2 Package SG039C — Cholecystectomy

Discharge summary detected on page 1 with clinical condition and pain present; page 2 is continuation of the same discharge summary (same document_rank = 5).

```
[
{
"case_id": "MAV/GJ/R3/2026032310030008",
"S3_link": "medclaim-bucket/cholecystectomy.pdf",
"procedure_code": "SG039C",
"page_number": 1,
"clinical_notes": 0, "usg_report": 0, "lft_report": 0,
"operative_notes": 0, "pre_anesthesia": 0,
"discharge_summary": 1,
"photo_evidence": 0, "histopathology": 0,
"clinical_condition": 1, "usg_calculi": 0,
"pain_present": 1, "previous_surgery": 0,
"extra_document": 0, "document_rank": 5
}
]
```

4.3 Package MG006A — Enteric Fever

Pre-investigation report found on pages 1 and 2 (same document, date extracted). Document quality flagged as poor; fever and symptoms not yet confirmed.


```
[
{
"case_id": "C1",
"S3_link": "medclaim-bucket/enteric_fever.pdf",
"procedure_code": "MG006A",
"page_number": 1,
"clinical_notes": 0,
"investigation_pre": 1,
"pre_date": "06-02-2026",
"vitals_treatment": 0,
"investigation_post": 0,
"post_date": null,
"discharge_summary": 0,
"poor_quality": 1,
"fever": 0,
"symptoms": 0,
"extra_document": 0,
"document_rank": 2
}
]
```


4.4 Package SB039A — Total Knee Replacement

Both uploaded pages are extra documents (unrelated to TKR package). DOA and DOD not found; all clinical keys absent.


```
[
{
"case_id": "C1",
"S3_link": "medclaim-bucket/tkr_claim.pdf",
"procedure_code": "SB039A",
"page_number": 1,
"clinical_notes": 0, "xray_ct_knee": 0,
"indoor_case": 0, "operative_notes": 0,
"implant_invoice": 0, "post_op_photo": 0,
"post_op_xray": 0, "discharge_summary": 0,
"doa": null, "dod": null,
"arthritis_type": 0,
"post_op_implant_present": 0,
"age_valid": 0,
"extra_document": 1,
"document_rank": 99
}
]
```

5. Database Schema (Supabase)

Table: claims

Column	Type	Description
id	UUID	Primary key — unique claim identifier
status	Text	pending / processing / completed / failed
data	JSONB	Full extracted claim data including all per-page JSON objects
file_names	Text[]	Array of uploaded document file names
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Last update timestamp

Table: chat_messages

Column	Type	Description
id	UUID	Primary key
claim_id	UUID	Foreign key → claims.id
role	Text	user or assistant
content	Text	Message content
created_at	Timestamp	Message timestamp

5.1 Key Design Decisions

Client-side PDF rasterisation: Avoids server load. pdfjs runs in the browser worker thread, producing JPEG images at 1.6x scale — optimal for vision model accuracy without excessive payload size.

Lazy Supabase client initialisation: Both client.ts and client.server.ts use a Proxy pattern to defer client creation until first use, preventing env-variable errors at module load time.

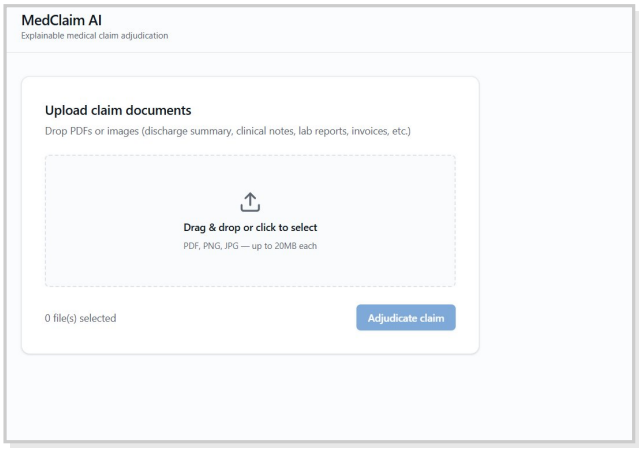
Deterministic rules engine: STG checks are coded as explicit TypeScript functions (not AI judgement), ensuring 100% reproducibility and auditability for each rule result.

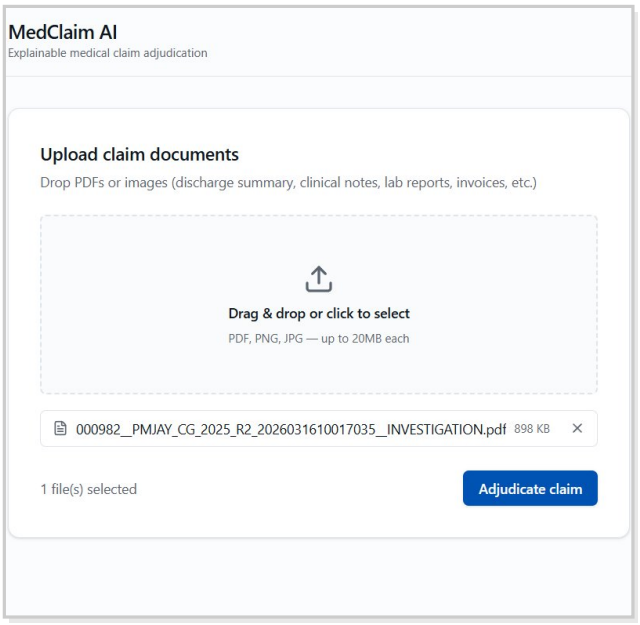
JSONB for claim data: Storing the full per-page JSON in a single JSONB column allows flexible querying across packages without schema migrations per package.

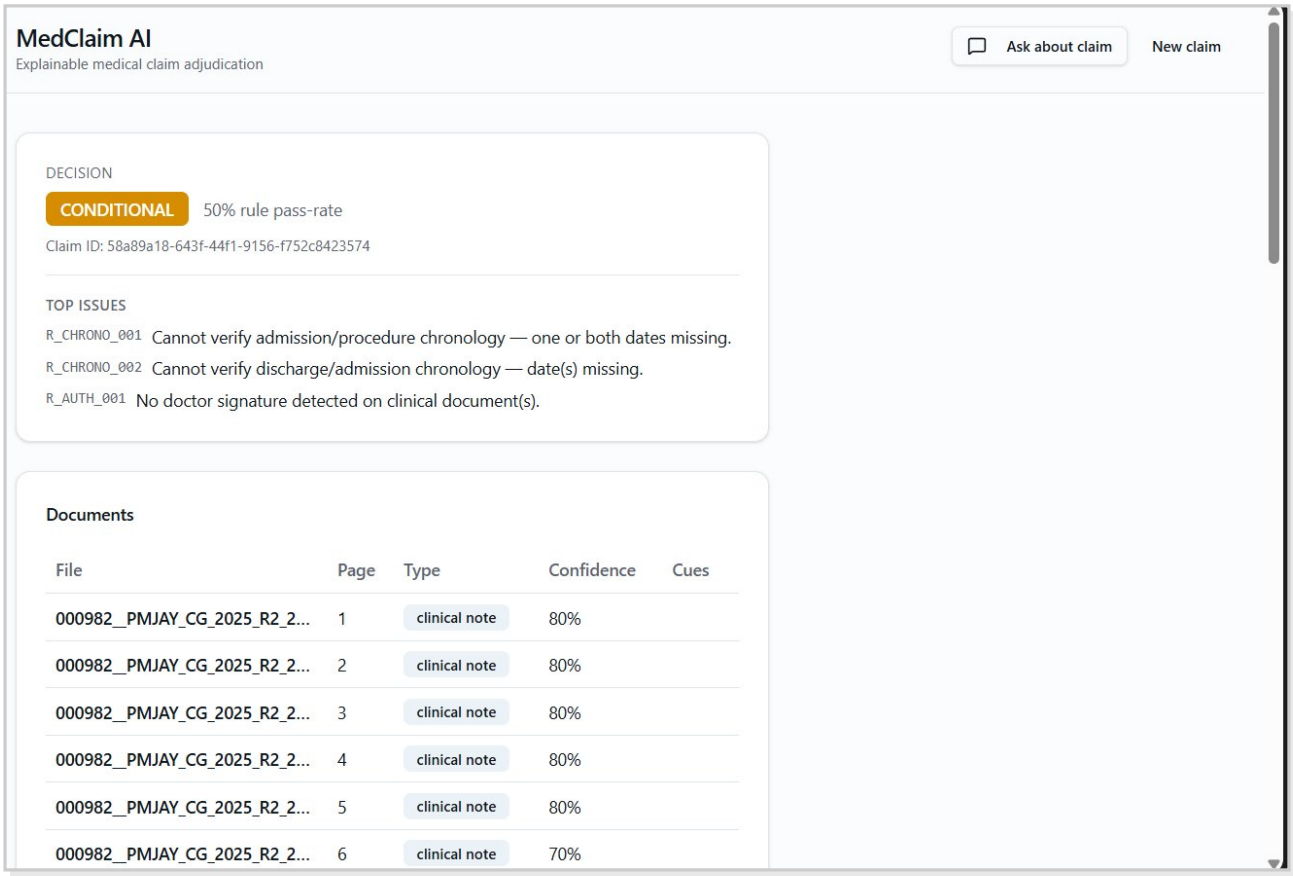
5.2 Verdict Scoring Logic

Condition	Score Contribution	Final Verdict
Any high-severity rule fails	+3 per failure	FAIL (immediate)
Medium-severity rule fails	+2 per failure	—

Any rule warns	+1 per warn	—
Total score ≥ 2, no high fail	—	CONDITIONAL
Total score < 2, no high fail	—	PASS







Snapshots of Website

MedClaim AI — Explainable Medical Claim Adjudication Interface

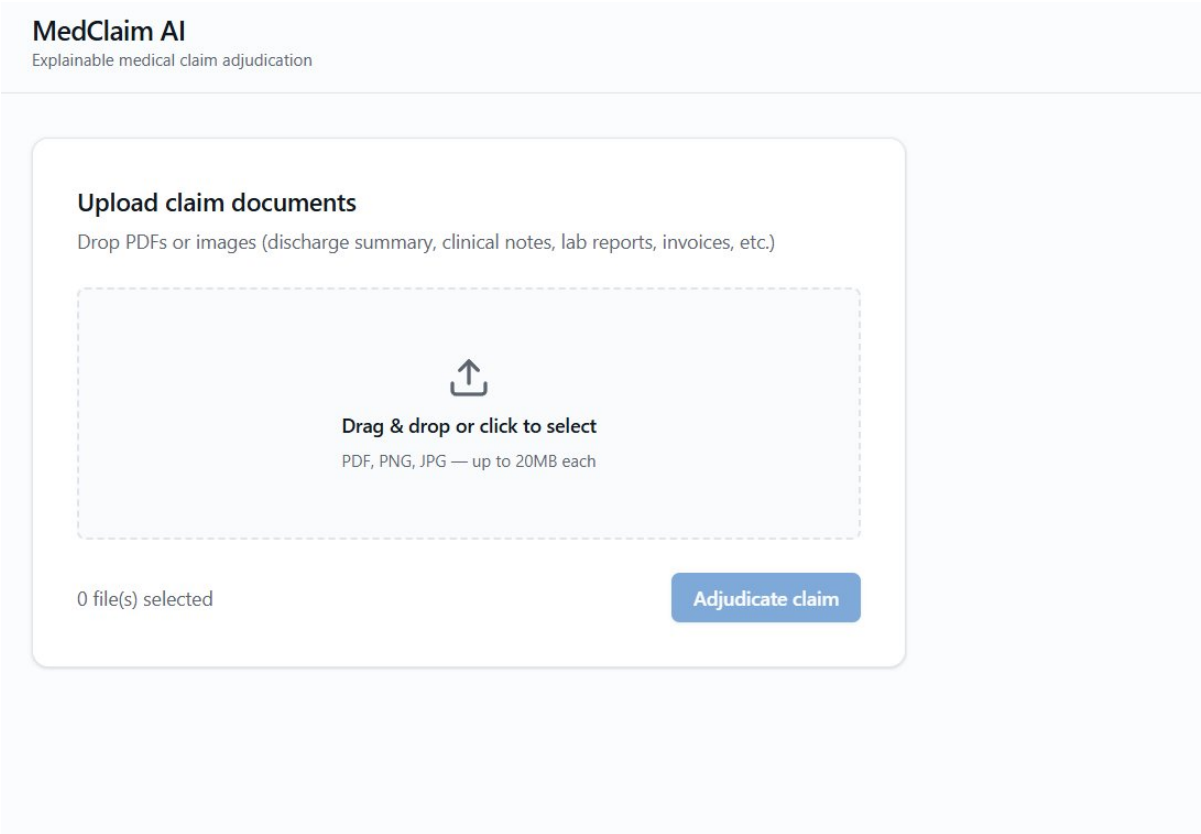


Fig 1 — Upload interface (empty state)

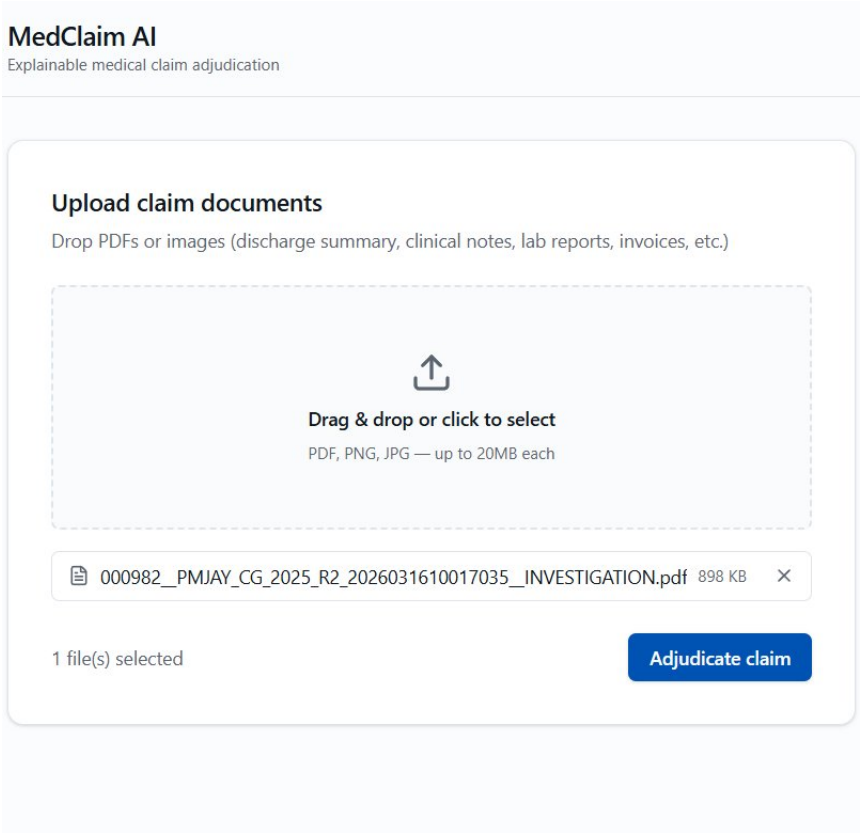


Fig 2 — File selected and ready for adjudication

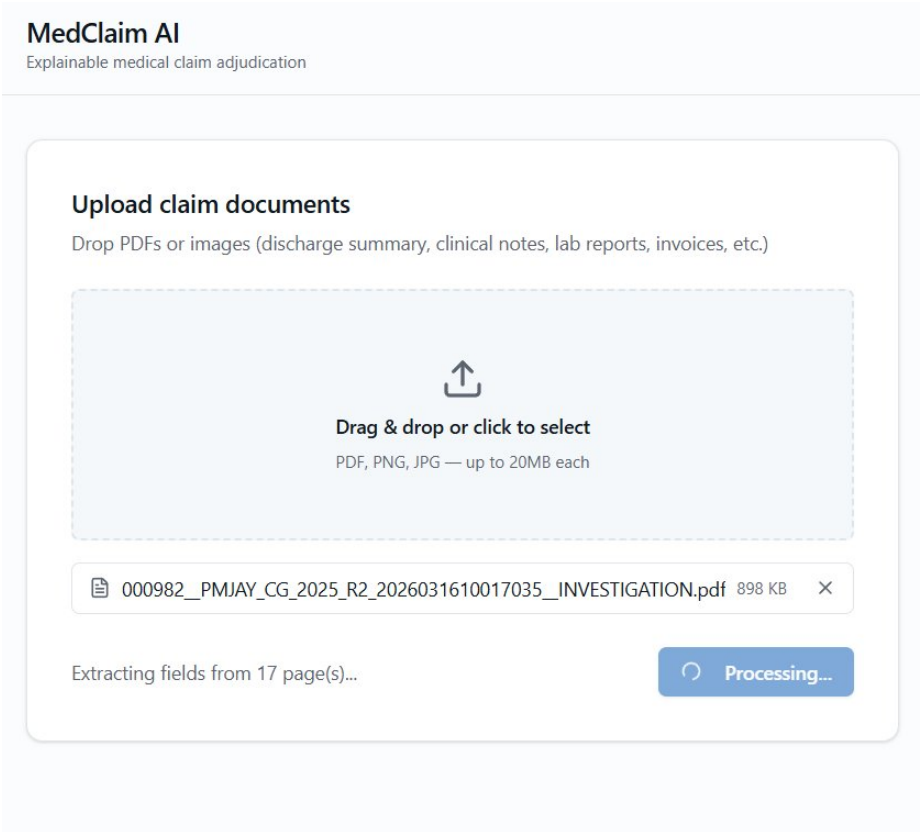


Fig 3 — Processing state: extracting fields from 17 pages

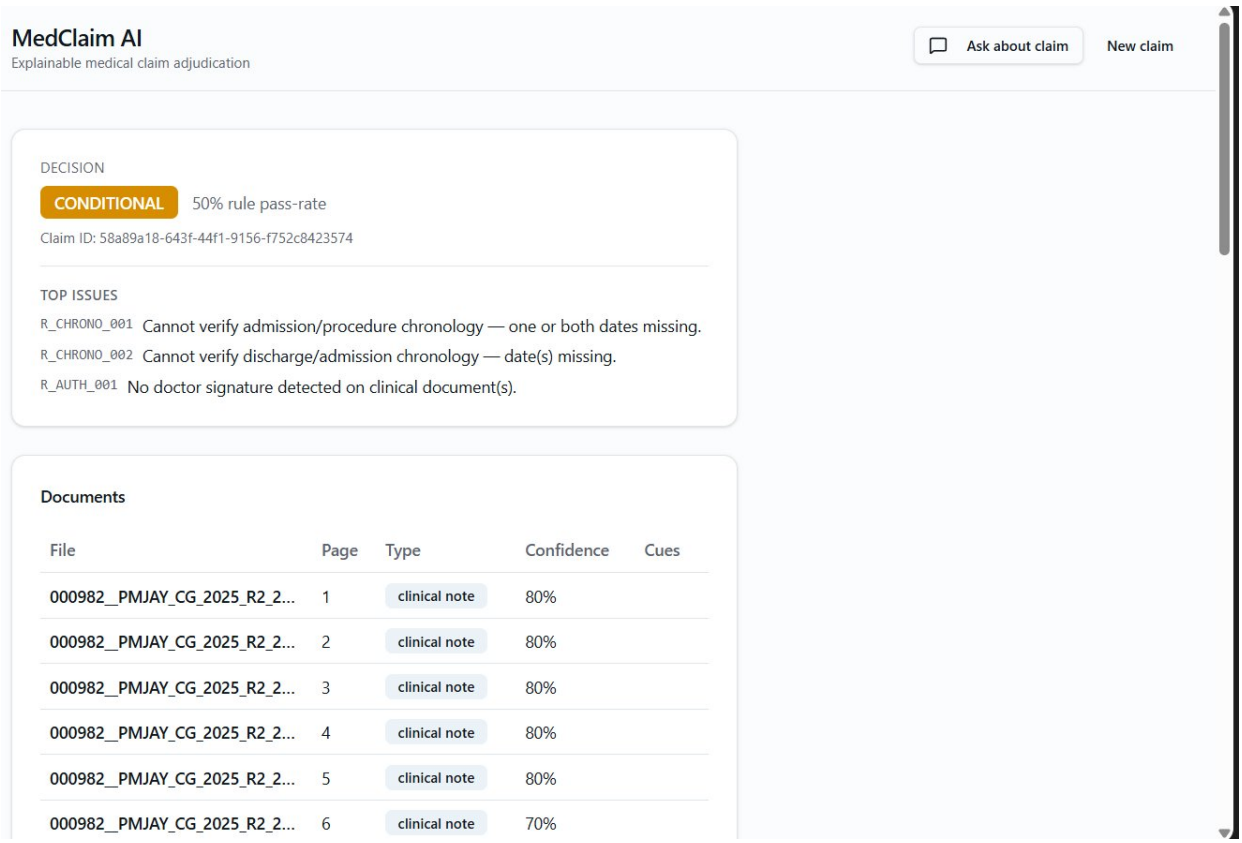


Fig 4 — Adjudication result: CONDITIONAL decision with top issues

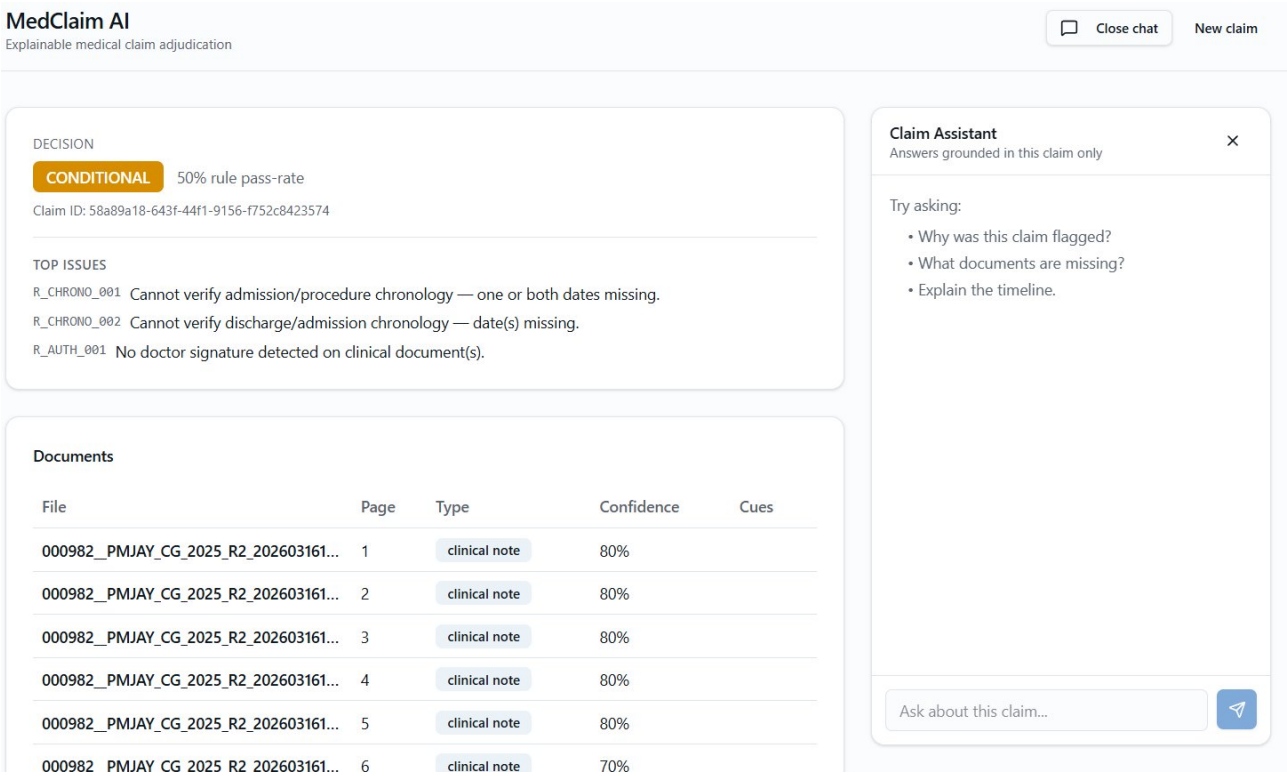


Fig 5 — Claim Assistant chat panel open alongside decision

Documents

File	Page	Type	Confidence	Cues
000982__PMJAY_CG_2025_R2_202603161...	1	clinical note	80%	
000982__PMJAY_CG_2025_R2_202603161...	2	clinical note	80%	
000982__PMJAY_CG_2025_R2_202603161...	3	clinical note	80%	
000982__PMJAY_CG_2025_R2_202603161...	4	clinical note	80%	
000982__PMJAY_CG_2025_R2_202603161...	5	clinical note	80%	
000982__PMJAY_CG_2025_R2_202603161...	6	clinical note	70%	
000982__PMJAY_CG_2025_R2_202603161...	7	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	8	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	9	lab report	90%	QR
000982__PMJAY_CG_2025_R2_202603161...	10	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	11	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	12	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	13	lab report	90%	
000982__PMJAY_CG_2025_R2_202603161...	14	lab report	80%	
000982__PMJAY_CG_2025_R2_202603161...	15	lab report	80%	
000982__PMJAY_CG_2025_R2_202603161...	16	lab report	90%	

Fig 6 — Documents table listing all 16 pages with type and confidence

Timeline

#	Event	Date	Source	Validity
1	**Procedure**	null	000982__PMJAY_CG_2025_R2_20260316100170...	Date missing
2	**Discharge**	null	000982__PMJAY_CG_2025_R2_20260316100170...	Date missing
3	**Admission**	2016-03-16	000982__PMJAY_CG_2025_R2_20260316100170...	Valid
4	**Diagnostic Investigation**	2026-03-15	000982__PMJAY_CG_2025_R2_20260316100170...	Valid

All rule checks

CRITICAL

R_CHRONO_001 WARN

Cannot verify admission/procedure chronology — one or both dates missing.

Source: —

R_CHRONO_002 WARN

Cannot verify discharge/admission chronology — date(s) missing.

Source: —

R_DOC_001 PASS

At least one clinical document is present.

Source: 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf,
000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf,
000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf,
000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf,
000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf,
000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf

Fig 7 — Timeline view and rule checks (R_CHRONO, R_DOC, R_KIE)

R_CHRONO_002 WARN

Cannot verify discharge/admission chronology — date(s) missing.

Source: —

R_DOC_001 PASS

At least one clinical document is present.

Source: 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf

R_KIE_001 PASS

Patient name extracted: "Mrs. Wijoy leamidas".

Source: 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf p.4

MODERATE

R_DIAG_001 PASS

Diagnostic investigation document is present.

Source: 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf

MINOR

R_AUTH_001 WARN

No doctor signature detected on clinical document(s).

Source: 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf, 000982__PMJAY_CG_2025_R2_2026031610017035__INVESTIGATION.pdf

Fig 8 — Detailed rule results: WARN and PASS statuses across severity levels